

# Pelican-VLA 0.5: Attending Before Acting Benefits Generalization

Beijing Innovation Center of Humanoid Robotics (X-Humanoid)

WFM System Group

{vito.dai,jian.tang,jason.ju}@x-humanoid.com

<https://github.com/Open-X-Humanoid/Pelican-VLA05>

<https://huggingface.co/X-Humanoid/Pelican-VLA05>

July 9, 2026

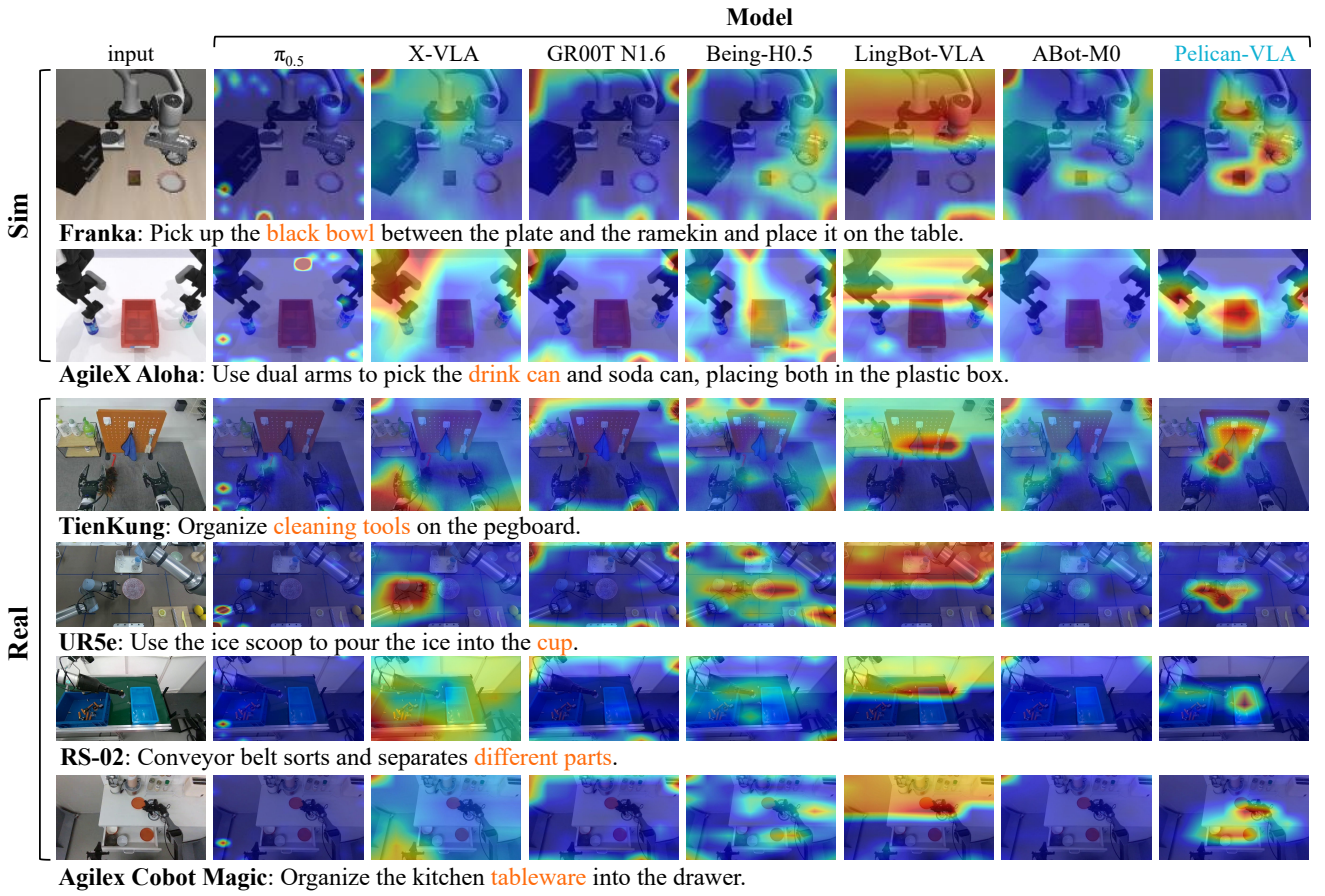


Figure 1: **Attention visualization comparison with open-source VLA baselines in the zero-shot setting.** Before task-specific fine-tuning, Pelican-VLA 0.5 directs its action-pathway attention to the instruction-relevant object and contact area. In contrast, other open-source VLA models show more diffuse attention, often spreading over the robot arm, surrounding objects, or background.

## Abstract

A central goal of Vision-Language-Action (VLA) research is to build robotic models that can truly generalize across objects, scenes, tasks, and embodiments. However, current VLA models still depend heavily on task- and environment-specific robot data, and typically require additional data collection and fine-tuning when transferred to new objects, scenes, tasks, or embodiments. Recent studies suggest that a VLA may first learn transferable internal representations of *what* to act upon, and only later learn to convert these representations into precise executable actions. This suggests that helping the action pathway attend more consistently to manipulation-relevant regions may be beneficial for generalization. Motivated by this view, we analyze



the attention of VLA models and find that their action pathways attend *diffusely*, spreading over the robot arm, background, and task-irrelevant objects.

In this report, we present **Pelican-VLA 0.5**, a unified VLA model that integrates vision-language understanding, future-frame generation, and action prediction within a single architecture. Pelican-VLA 0.5 achieves *attention-level generalization*: without object annotations, segmentation masks, attention supervision, or task-specific fine-tuning, its action pathway already focuses on the instruction-relevant object and contact region. This behavior persists across unseen scenes and unseen robot embodiments, and is substantially stronger than in other open-source VLA baselines. We verify that this ability originates from the learnable **Reasoning Slots** inserted between perception and action: by routing task-relevant visual information through a compact bottleneck, the slot interface induces manipulation-centric attention during pre-training and remains effective across different policy structures, including a MoT-style architecture.

After fine-tuning on RoboTwin, Pelican-VLA 0.5 achieves **91.4%** success on *RoboTwin Clean* and **91.0%** on *RoboTwin Randomized*, the best average among open-source VLA baselines. In zero-shot settings, including unseen scenes, unseen objects, and new robot embodiments, Pelican-VLA 0.5 attains non-zero success on several tasks, an early glimmer of generalization. Notably, the attention patterns before and after fine-tuning remain highly similar, suggesting that fine-tuning mainly strengthens the mapping from these pre-formed, manipulation-centric attention regions to executable actions, rather than creating them from scratch.

These findings clarify the meaning of **Pelican-VLA 0.5**. The model has already achieved strong attention-level generalization during pre-training, but an attention-to-action gap still remains. It can begin to identify *what* to attend to and represent, while reliable execution across new scenes, objects, and embodiments still requires stronger action-level generalization. Pelican-VLA 0.5 is an intermediate model toward truly generalizable VLA models.

## 1. Introduction

Building robots that can follow free-form language instructions and manipulate objects in open, unstructured environments is a central goal of embodied intelligence. Vision-Language-Action (VLA) models have become a leading paradigm toward this goal [37, 15, 20, 7, 18, 23, 5]. By learning an end-to-end mapping from visual observations and language instructions to robot actions, and by building on large-scale multimodal pre-training [21, 14, 6, 33], these models inherit broad semantic priors and increasingly demonstrate strong performance on everyday manipulation tasks. However, the defining challenge for VLA models is not solving a single benchmark after sufficient adaptation, but *generalization*: a useful robot policy should behave sensibly when the target object, scene, task, or even robot embodiment differs from those seen during training. Despite rapid progress, reliable zero-shot manipulation in unfamiliar settings remains difficult [15, 6, 32, 33].

Recent studies [6, 33, 9] suggest that VLA generalization depends not only on fitting low-level action trajectories, but also on how task-relevant information is represented and routed before action prediction. Motivated by this perspective, we analyze the attention maps of representative VLA models. We find that their action pathways often attend *diffusely* over the visual input, with attention spread across the robot arm, background clutter, and task-irrelevant objects, rather than consistently concentrating on the instruction-relevant target region or potential contact area. This observation is consistent with recent findings that action decoders in VLA models do not inherently learn to extract task-critical information and may instead rely on visual shortcuts or environmental noise [12]. It motivates us to ask whether a more constrained interface between perception and action can encourage the model to distill manipulation-relevant information more effectively, thereby supporting stronger generalization across new scenes, objects, and embodiments.

In this report, we study this question through **Pelican-VLA 0.5**, a unified VLA model that integrates vision-language understanding, future generation, and action prediction within a single Transformer backbone. Pelican-VLA 0.5 exhibits strong *attention-level generalization*: before any task-specific finetuning, its action pathway already concentrates on instruction-relevant target regions and contact areas. A central design is a compact set of learnable **Reasoning Slots** inserted between perception and action. These slots serve as a learnable bottleneck between perception and action, querying the upstream vision-language context through full attention and condensing manipulation-relevant information before passing it to the action pathway. Rather than allowing the action pathway to directly attend to dense visual tokens, these slots provide a constrained interface through which task-relevant perceptual information is routed to action generation. This slot-mediated interface encourages the model to organize manipulation-relevant information, including the target object, potential contact region, intended future change, and current robot-state constraint, while reducing direct reliance on low-level visual details.

In the zero-shot setting, Pelican-VLA 0.5’s action pathway already concentrates on the instruction-relevant object and contact region, as shown in Fig. 1. This behavior emerges without object-level supervision and persists across unseen scenes and unseen robot embodiments. Compared with other open-source VLA baselines, Pelican-VLA 0.5 exhibits substantially more target-aligned attention. Moreover, removing the slots after training does not fully eliminate the model’s ability to attend to the instruction-relevant object, indicating that the slot bottleneck helps internalize manipulation-centric attention into the shared backbone and allows the learned representation to persist as a property of the trained model itself.

In RoboTwin simulation, Pelican-VLA 0.5 can perform zero-shot attempts in unseen scenes and new embodiments, as shown in Fig. 3. We fine-tune Pelican-VLA 0.5 on RoboTwin. After fine-tuning, the model achieves strong average success among open-source VLA models, reaching **91.4%** on *RoboTwin Clean* and **91.0%** on *RoboTwin Randomized*. These results show that the pre-formed manipulation-centric attention can be effectively converted into task success once sufficient action supervision is provided. More importantly, when we visualize the attention maps before and after fine-tuning, we find that the attention patterns remain highly similar, as shown in Fig. 10. This suggests that fine-tuning Pelican-VLA 0.5 does not primarily create manipulation-centric attention from scratch. Instead, it mainly improves the mapping from manipulation-centric attentions to executable actions.

These findings clarify the meaning of **Pelican-VLA 0.5**. The model is not presented as a fully zero-shot VLA. Rather, it represents an intermediate stage toward truly generalizable robot policies. It has achieved strong representation-level generalization during pre-training, but a representation-to-action gap still remains. This limitation is partly due to the scale and form of the current training data: Pelican-VLA 0.5 is trained on 2,400 hours of heterogeneous robot data and uses joint-position actions, which are more embodiment-specific than end-effector action representations. Scaling robot data, improving action representations, and strengthening the representation-to-action mapping are therefore necessary next steps toward practical zero-shot manipulation.

### Contributions.

- We introduce **Pelican-VLA 0.5**, a unified VLA architecture that integrates vision-language understanding, future generation, and action prediction within a shared Transformer backbone. By inserting learnable Reasoning Slots between perception and action, Pelican-VLA 0.5 encourages the action pathway to extract and route manipulation-relevant information before action generation
- We identify a form of **attention-level generalization** that emerges during pre-training and remains stable across zero-shot evaluation, downstream fine-tuning, and architectural transfer. Without task-specific fine-tuning, object annotations,

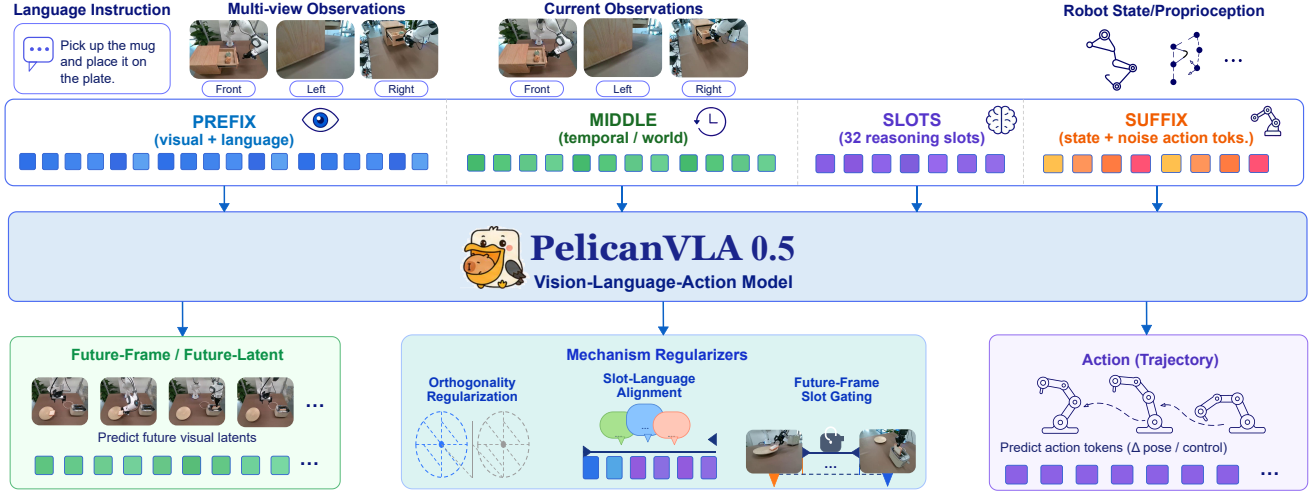


Figure 2: **Overview of Pelican-VLA 0.5.** Pelican-VLA 0.5 uses a shared Qwen3-VL 4B backbone to unify visual-language understanding, future-frame prediction, and action generation within a single Transformer. The input sequence contains four segments: vision-language *prefix* tokens, Cosmos latent *middle* tokens,  $K$  learnable *reasoning slots*, and a *suffix* containing proprioceptive states and noisy action chunks. A single forward pass predicts future-frame latents from the middle, a compact task representation from the slots, and action denoising velocities from the suffix. The key design is a bottleneck attention mask: the action pathway reads perception through the reasoning slots rather than directly over dense visual tokens. This encourages manipulation-centric attention and allows the denoising loop to reuse a compact slot cache during inference.

segmentation masks, or attention supervision, Pelican-VLA 0.5 already attends to instruction-relevant target regions and contact areas across unseen scenes and unseen robot embodiments.

- We verify that this representation ability originates from learnable *Reasoning Slots* inserted between perception and action. The slot interface induces manipulation-centric attentions by routing task-relevant visual information through a compact bottleneck, and remains effective across different policy structures, including a MoT-style architecture.
- We clarify the gap between zero-shot representation and zero-shot control. Pelican-VLA 0.5 shows partial zero-shot manipulation ability, and after RoboTwin fine-tuning achieves **91.4%** success on *RoboTwin Clean* and **91.0%** on *RoboTwin Randomized*; we will release the code, pretrained and fine-tuned weights, and attention-visualization tools to support reproducibility.

## 2. Pelican-VLA 0.5

### 2.1. Overview

As shown in Fig. 2, Pelican-VLA 0.5 is a unified Vision-Language-Action model built on a single shared Qwen3-VL 4B backbone [4]. The model is designed to support three tightly coupled functions within one Transformer: visual-language understanding, future-frame prediction, and action generation. Unlike dual-system or Mixture-of-Transformers designs [8, 19], Pelican-VLA 0.5 does not introduce separate reasoning and acting experts. Instead, all functions operate over a shared token stream and communicate through a common hidden state.

The input sequence is organized into four contiguous segments:

$$\mathbf{Z} = \left[ \underbrace{\text{Prefix}}_{\text{vision-language}} ; \underbrace{\text{Middle}}_{\text{Cosmos latents}} ; \underbrace{\text{Slots}}_{K \text{ reasoning slots}} ; \underbrace{\text{Suffix}}_{\text{state and noisy action}} \right]. \quad (1)$$

A single forward pass produces three outputs in parallel: action denoising velocities from the suffix, future-frame latents from the middle, and a compact task representation from the slots.

This section first describes the unified token sequence and attention geometry in §2.2. We then introduce the reasoning-slot bottleneck and its auxiliary regularizers in §2.3, followed by the training objectives in §2.4 and the cached inference procedure in §2.5.

### 2.2. Unified token sequence

**Prefix: vision-language tokens.** The current camera observations and language instruction are encoded by the Qwen3-VL backbone. Each camera frame is first processed by the Qwen3-VL vision encoder, and the resulting visual patch embeddings



are inserted into the text-token sequence at the corresponding image-token positions. This produces the prefix sequence

$$\mathbf{P} \in \mathbb{R}^{L_p \times D}. \quad (2)$$

The prefix uses bidirectional attention so that language tokens and visual tokens can mutually contextualize each other. This segment provides the semantic visual-language representation of the current scene.

**Middle: Cosmos latent tokens.** In parallel, we encode a short visual history using a frozen Cosmos-Tokenizer [2]. For each view, two frames at time  $t-15$  and  $t$  are mapped into continuous latent features. The features are flattened into the middle sequence

$$\mathbf{M} \in \mathbb{R}^{L_m \times D}. \quad (3)$$

The middle segment provides a pixel-level and dynamics-aware view of the scene, complementing the semantic patch tokens in the prefix. It attends bidirectionally to itself and to the prefix, and is supervised by the future-frame prediction objective described in §2.4.

**Slots: learnable reasoning tokens.** We introduce  $K$  learnable reasoning slots,

$$\mathbf{S}_0 \in \mathbb{R}^{1 \times K \times D}, \quad (4)$$

initialized from  $\mathcal{N}(0, 0.02^2)$  and broadcast across the batch. During training, slot dropout with probability  $p = 0.1$  is applied:

$$\mathbf{S} = \text{drop}_p(\mathbf{S}_0) \in \mathbb{R}^{B \times K \times D}. \quad (5)$$

We use  $K = 32$  by default. Since  $K$  is much smaller than the number of upstream visual tokens, the slots form a fixed-capacity interface between perception and action. Their attention to the upstream context provides a direct way to inspect what information is routed into the action pathway.

**Suffix: proprioceptive state and noisy action.** The suffix contains the robot state and the noisy action chunk used by the flow-matching. The proprioceptive state is projected into a single state token. The noisy action chunk

$$\mathbf{x}_\tau \in \mathbb{R}^{H \times d_a} \quad (6)$$

is embedded by a linear layer, concatenated with a sinusoidal embedding of the flow time  $\tau$ , and fused by a two-layer MLP into  $H$  action tokens. We use  $H = 50$  and  $d_a = 32$  in our implementation. The suffix sequence is therefore

$$\mathbf{U} \in \mathbb{R}^{(1+H) \times D}. \quad (7)$$

The state token and action tokens are treated as separate attention blocks: the state token provides the current proprioceptive condition, while the action tokens are decoded under the visible context.

**Attention geometry.** The full sequence is

$$\mathbf{Z} = [\mathbf{P}; \mathbf{M}; \mathbf{S}; \mathbf{U}]. \quad (8)$$

We construct a two-dimensional attention mask from per-token block flags, following the prefix-LM and block-causal construction used in prior VLA architectures [7]. The prefix, middle, and slot segments are internally bidirectional. The suffix uses causal or block-causal attention with respect to its visible context.

When `enable_reasoning_slots=true`, the slot segment is inserted between the upstream perception tokens and the action suffix. Disabling the slots recovers a standard dense-attention VLA in which the action suffix can directly attend to the upstream visual context. We use this dense variant as the primary architectural ablation.

### 2.3. The Reasoning-Slot bottleneck

The reasoning slots are designed not only as an information bottleneck, but also as an implicit task-reasoning workspace. The reasoning slots become an effective bottleneck only when the action suffix cannot bypass them. Pelican-VLA 0.5 enforces this bottleneck through three mechanisms: a curriculum attention mask, an orthogonality regularizer, and a slot-gated generation.

**Curriculum bottleneck mask.** Let  $\mathcal{C} = \{\text{prefix}\} \cup \{\text{middle}\}$  denote the upstream perception tokens, and let  $\mathcal{S}$  denote the reasoning slots. We define a binary attention-visibility mask  $A$ , where  $A_{i \rightarrow j} = 1$  means that query token  $i$  is allowed to attend to key/value token  $j$ , and  $A_{i \rightarrow j} = 0$  means that this attention edge is masked out. In the hard bottleneck regime, we remove all direct attention edges from suffix tokens to the upstream perception tokens:

$$A_{i \rightarrow j} = 0, \quad \forall i \in \text{suffix}, j \in \mathcal{C}. \quad (9)$$

The suffix can still attend to the slots and to its permitted suffix context. Thus, the only path from perception to action passes through  $\mathcal{S}$ .

Applying this hard constraint from the beginning of training can make optimization unstable. We therefore use a curriculum schedule. At training step  $t$ , the hard bottleneck mask is applied with probability

$$p_t = \min\left(1, \frac{t}{T_{\text{warm}}}\right), \quad T_{\text{warm}} = 10k. \quad (10)$$

With probability  $1 - p_t$ , the suffix is allowed to directly attend to the upstream context. This schedule lets the model first learn with an easier dense channel and then progressively shifts perception-to-action communication through the slots.

**Orthogonality regularizer.** A compact slot set is only useful if different slots capture complementary information. Let  $\mathbf{Z}^{\text{slot}} \in \mathbb{R}^{B \times K \times D}$  be the final slot outputs of the Transformer. For each batch element  $b$ , let  $\hat{\mathbf{Z}}_b^{\text{slot}} \in \mathbb{R}^{K \times D}$  denote the slot-output matrix after  $\ell_2$ -normalizing each slot vector along the hidden dimension. We encourage slot diversity by penalizing deviations of the slot Gram matrix from identity:

$$\mathcal{L}_{\text{reg}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{K^2} \left\| \hat{\mathbf{Z}}_b^{\text{slot}} (\hat{\mathbf{Z}}_b^{\text{slot}})^\top - \mathbf{I}_K \right\|_F^2. \quad (11)$$

Here  $\mathbf{I}_K$  is the  $K \times K$  identity matrix and  $\|\cdot\|_F$  denotes the Frobenius norm. This prevents all slots from collapsing to the same direction and encourages them to encode distinct factors of the scene, task, or action context.

**Slot gating of generation.** The slots are also connected to the future-frame prediction branch. From the mean-pooled slot output

$$\bar{\mathbf{z}} = \frac{1}{K} \sum_{k=1}^K \mathbf{z}_k^{\text{slot}}, \quad (12)$$

we compute a channel-wise gate  $\mathbf{g} = \sigma(\mathbf{W}_g \bar{\mathbf{z}})$ , where  $\mathbf{W}_g$  projects the pooled slot feature to the channel dimension of the generation features. The gate modulates the generation features before they are decoded back into Cosmos latent space:

$$\tilde{\mathbf{F}}_{\text{gen}} = \mathbf{F}_{\text{gen}} \odot \mathbf{g}. \quad (13)$$

Here  $\mathbf{g}$  is broadcast over the token or spatial dimensions of  $\mathbf{F}_{\text{gen}}$ . This encourages the slots to contain information that is useful not only for action prediction, but also for predicting what will happen next. In this way, the slot representation is tied to forward-predictive, action-relevant structure rather than static appearance alone.

## 2.4. Training objectives

Pelican-VLA 0.5 is trained with a multi-task objective that combines action generation, future-frame prediction, language-task alignment, and slot regularization.

**Flow-matching action loss.** We use conditional flow matching [16, 7] to model the action chunk. Let  $\mathbf{a} \in \mathbb{R}^{H \times d_a}$  be the target delta-action chunk, where  $H$  is the action horizon and  $d_a$  is the action dimension. We sample Gaussian noise  $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  with the same shape as  $\mathbf{a}$ , and sample the flow time  $\tau \sim \mathcal{U}(0, 1)$ . We construct the interpolated noisy action and the target velocity as

$$\mathbf{x}_\tau = \tau \boldsymbol{\varepsilon} + (1 - \tau) \mathbf{a}, \quad \mathbf{u} = \boldsymbol{\varepsilon} - \mathbf{a}. \quad (14)$$

Here  $\mathbf{x}_\tau$  moves from the clean action  $\mathbf{a}$  at  $\tau = 0$  to Gaussian noise  $\boldsymbol{\varepsilon}$  at  $\tau = 1$ , and  $\mathbf{u}$  is the constant velocity along this linear path. The noisy action  $\mathbf{x}_\tau$  is embedded into the suffix, and the action-token outputs are passed through a LayerNorm-MLP head  $v_\theta$ . The action loss is

$$\mathcal{L}_{\text{action}} = \mathbb{E}_{\mathbf{a}, \boldsymbol{\varepsilon}, \tau} \left\| v_\theta(\mathbf{Z})_{\text{act}} - \mathbf{u} \right\|_2^2. \quad (15)$$

Here  $v_\theta(\mathbf{Z})_{\text{act}} \in \mathbb{R}^{H \times d_a}$  denotes the decoded outputs corresponding only to the  $H$  action tokens.

**Future-frame generation loss.** The middle outputs are decoded by the slot-gated generation head and supervised in the frozen Cosmos latent space. Given the future frame  $\mathbf{I}_{t+15}$ , we first encode it into a target Cosmos latent  $\mathbf{c}_{t+15} = \text{Cosmos}(\mathbf{I}_{t+15})$ . We compute

$$\mathcal{L}_{\text{gen}} = \|\text{Dec}(\tilde{\mathbf{F}}_{\text{gen}}) - \mathbf{c}_{t+15}\|_2^2. \quad (16)$$

Here  $\text{Dec}(\cdot)$  denotes the latent prediction head that maps the gated generation features to the Cosmos latent space. The loss is computed over valid cameras only. Predicting in Cosmos latent space provides a compact dynamics signal and avoids the cost and instability of direct pixel reconstruction [30].

**Slot-language contrastive alignment.** We further align the trajectory representation with the language instruction using a symmetric InfoNCE objective [26, 24, 17]. For a mini-batch of size  $B$ , let  $\mathbf{Z}_{\text{traj}} \in \mathbb{R}^{B \times d_c}$  denote the trajectory embeddings obtained by mean-pooling and projecting the selected trajectory feature source, and let  $\mathbf{Z}_{\text{text}} \in \mathbb{R}^{B \times d_c}$  denote the text embeddings obtained by mean-pooling the instruction token embeddings. Both embeddings are  $\ell_2$ -normalized along the embedding dimension, where  $d_c$  is the contrastive embedding dimension. With learnable temperature  $\exp(s)$ , the batchwise similarity logits are

$$\mathbf{L} = \exp(s) \mathbf{Z}_{\text{traj}} \mathbf{Z}_{\text{text}}^\top. \quad (17)$$

The contrastive loss is

$$\mathcal{L}_{\text{task}} = \frac{1}{2} (\text{CE}(\mathbf{L}, \mathbf{y}) + \text{CE}(\mathbf{L}^\top, \mathbf{y})), \quad y_i = i, i = 1, \dots, B. \quad (18)$$

Here  $\mathbf{y}$  is the identity matching target within the mini-batch: the  $i$ -th trajectory is paired with the  $i$ -th instruction. When different samples share the same instruction, the corresponding off-diagonal pairs are masked out of the denominator to avoid treating true semantic matches as negatives.

**Total objective.** The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda_{\text{gen}} \mathcal{L}_{\text{gen}} + \lambda_{\text{task}} \mathcal{L}_{\text{task}} + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (19)$$

In our pre-training runs, we set  $\lambda_{\text{gen}} = 0.01$ ,  $\lambda_{\text{task}} = 0.1$ ,  $\lambda_{\text{reg}} = 0.01$ . The bottleneck schedule is advanced by the global training step, so the perception-to-action pathway is gradually constrained as optimization proceeds.

## 2.5. Cached inference

At inference time, the reasoning slots provide a compact cacheable interface for action denoising. The prefix and middle tokens depend only on the current visual-language context, not on the flow time. Therefore, they are computed once and cached. Inference proceeds in three stages.

First, the prefix is processed by the language model with key-value caching enabled. Second, the middle tokens and reasoning slots extend the cache. After this stage, we keep only the  $K$  slot key-value pairs as the visual context for the denoising loop and discard the much larger prefix and middle cache. Third, we initialize the action chunk from Gaussian noise,  $\mathbf{x}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ , and integrate the flow ODE backward from  $\tau = 1$  to  $\tau = 0$  using  $N = 10$  Euler steps. At each step, only the suffix tokens are re-embedded and passed through the model using a copy of the slot cache:

$$\mathbf{x}_{\tau+\Delta\tau} = \mathbf{x}_\tau + \Delta\tau \cdot v_\theta(\mathbf{Z})_{\text{act}}, \quad \Delta\tau = -\frac{1}{N}. \quad (20)$$

Here  $v_\theta(\mathbf{Z})_{\text{act}} \in \mathbb{R}^{H \times d_a}$  denotes the predicted velocity for the  $H$  action tokens. After the final step reaches  $\tau = 0$ , the predicted delta actions are de-normalized and added to the current proprioceptive state. This procedure makes the repeated denoising cost depend on the suffix length and the number of slots, rather than on the full visual context length. As a result, the model can preserve a slot-mediated perception-to-action interface while supporting efficient high-frequency control.

## 3. Experiments

### 3.1. Training data

Pelican-VLA 0.5 is pre-trained on a heterogeneous, cross-embodiment mixture of large-scale manipulation corpora, including AgiBot World Alpha [3], InternData-A1 [25], the Galaxea Open-World Dataset [13], and roughly 1000 hours of self-collected teleoperation Tienkung and UR data whose collection format is comparable to RoboMIND [28]. The pre-training mixture contains more than 6000 hours of manipulation data. All datasets are unified through the LeRobot data interface. Since the corpora cover different robot embodiments with different degrees of freedom, we pad all state and action vectors to a common 32-dimensional format. This allows the model to train jointly across heterogeneous embodiments while using a shared action head. Unless otherwise specified, actions are represented in joint-position space.

Table 1: Benchmark results on seen tasks in RoboTwin. Compared with representative recent VLA baselines, Pelican-VLA 0.5 achieves the best performance in both clean and randomized settings, yielding the highest average success rate. Unless otherwise specified, best results are highlighted in **bold**, and second-best results are underlined.

| Methods                | Clean       | Randomized  | Average     |
|------------------------|-------------|-------------|-------------|
| $\pi_0$ [7]            | 80.0        | 79.5        | 79.8        |
| $\pi_{0.5}$ [6]        | 86.8        | 87.0        | 86.9        |
| X-VLA [36]             | 72.9        | 72.8        | 72.9        |
| StarVLA-OFT [11]       | 88.2        | 88.3        | 88.3        |
| ABot-M0 [31]           | 86.1        | 85.1        | 85.6        |
| LingBot-VLA [29]       | 88.6        | 86.7        | 87.7        |
| Qwen-VLA [27]          | 86.1        | 87.2        | 86.7        |
| JoyAI-RA [35]          | 90.5        | 89.3        | 89.9        |
| Hy-VLA [34]            | <u>90.9</u> | <u>90.1</u> | <u>90.5</u> |
| <b>Pelican-VLA 0.5</b> | <b>91.4</b> | <b>91.0</b> | <b>91.2</b> |

### 3.2. Implementation and pre-training recipe

We instantiate the backbone as the Qwen3-VL 4B and train it end-to-end in bfloat16 with gradient checkpointing. The action chunk length is  $H=50$  with delta-action targets, and the state and action vectors are padded to 32 dimensions for cross-embodiment training. Each observation comprises three camera views resized to  $224 \times 224$ , with image deltas of  $[-15, 0, 15]$ . Optimization uses AdamW with a peak learning rate of  $5 \times 10^{-5}$ , 2k warmup steps, and cosine decay to  $5 \times 10^{-6}$ , together with a per-GPU batch size of 6. The reasoning-slot module uses  $K=32$  slots with dropout 0.1 and a curriculum warmup of 10k steps, and the InfoNCE term is enabled with a task-embedding dimension of 256. The current model is pre-trained for only 0.4 epoch over this mixture, so it has effectively seen about 2400 hours of data; the results reported here are therefore obtained under a substantially under-trained regime.

### 3.3. Simulation results

On the RoboTwin benchmark [10], Pelican-VLA attains an average success rate of **91.4%** under the *clean* setting and **91.0%** under the *randomized* setting, as summarized in Table 1. These results are obtained with our proposed adaptive action execution strategy, which dynamically adjusts action execution according to the model’s prediction uncertainty. The gap between the two settings is only 0.4 points, indicating that the policy’s competence does not rely on the nominal appearance of the scene. This is consistent with our hypothesis that routing perception through a compact slot bottleneck suppresses low-level visual shortcuts and instead encodes task-relevant, object-centric structure that transfers across heavy randomization.

**Zero-shot generalization.** Beyond the in-distribution benchmark above, we probe whether Pelican-VLA can act *without any task-specific fine-tuning*. We deploy the pre-trained policy directly on RoboTwin 2.0, which is entirely held out from pre-training, under conditions it has never seen during training, including novel objects, novel scene layouts, and an *new robot embodiment*. To avoid inflating zero-shot performance through task-level leakage, we first audit the overlap between InternData-A1 and RoboTwin 2.0. We compute TF-IDF cosine similarity between task prompts from InternData-A1 and RoboTwin 2.0, manually re-examine all candidate pairs by reading the task descriptions and, when necessary, the corresponding demonstration videos, and exclude all tasks judged to be highly or moderately suspicious overlaps.

As shown in Fig. 3, Pelican-VLA exhibits early zero-shot manipulation behavior on tasks such as picking up bottle, placing a toy car onto a platform, picking up a beverage bottle, and turning on a switch. The policy reaches toward the correct instruction-relevant object and produces coherent, goal-directed motions rather than random or degenerate behavior, indicating that the manipulation-centric attention formed during pre-training transfers to unseen objects, scenes, and embodiments. However, this behavior remains imperfect: success rates in this strict zero-shot regime are still low, and failures typically occur at fine-grained stages such as stable grasping and precise placement rather than target selection or approach. This exposes the current representation-to-action gap in Pelican-VLA: learning to *see* the target is already emerging, while learning to *act* on it reliably remains the outstanding challenge. We believe this remaining gap is primarily data-driven, arising from the limited scale of the training data and the relatively weak cross-embodiment generalization of joint-space action representations, rather than from the slot bottleneck itself. As such, we expect the gap to be reduced through scaling robot data and improving action parameterizations, without changing the core slot-mediated architecture.

### 3.4. Real-world Robot Evaluation

**Tabletop cleanup on TienKung.** We further evaluate Pelican-VLA 0.5 on a real-world tabletop-cleanup task using the TienKung humanoid [22], as shown in Fig. 4. In this task, the robot must clear assorted objects from a cluttered desktop





Figure 3: **Zero-shot rollouts before task-specific fine-tuning.** Pelican-VLA 0.5 is deployed directly on RoboTwin in unseen scenes. The policy often approaches the instruction-relevant object and produces coherent task-directed motions, including pick up bottle, beverage-bottle grasping, and switch activation.

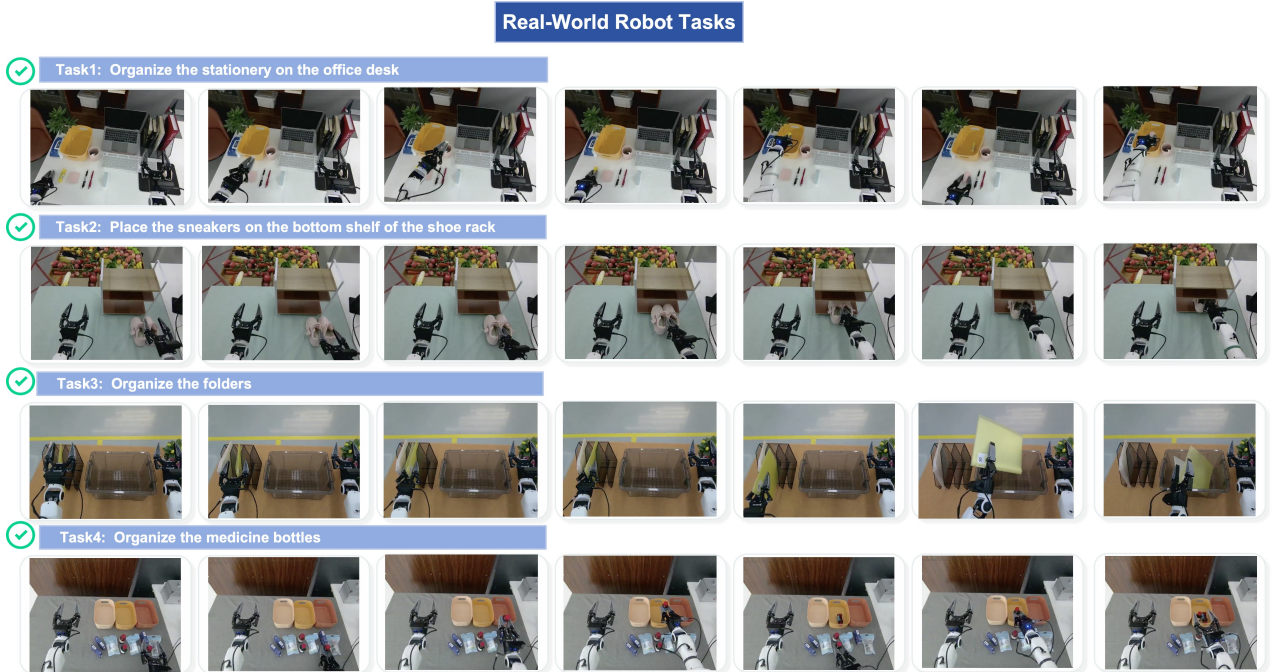


Figure 4: **Real-world tabletop cleanup on the TienKung humanoid.** Pelican-VLA 0.5 is fine-tuned on teleoperated demonstrations and deployed on a cluttered tabletop-cleanup task.

and place them into a designated container. The task requires long-horizon manipulation, repeated object selection, stable grasping, and robust execution under real-world sensing and actuation noise.

We fine-tune the pre-trained policy on teleoperated demonstrations collected on the TienKung platform and evaluate it over a fixed set of trials with randomized object types, object counts, and initial placements. This experiment tests whether the pre-trained representation can support downstream adaptation beyond simulation. As shown in Fig. 4, Pelican-VLA 0.5 completes the tabletop-cleanup task with a success rate of 80%, consistently selecting the correct objects, executing stable grasps, and placing them into the target container.

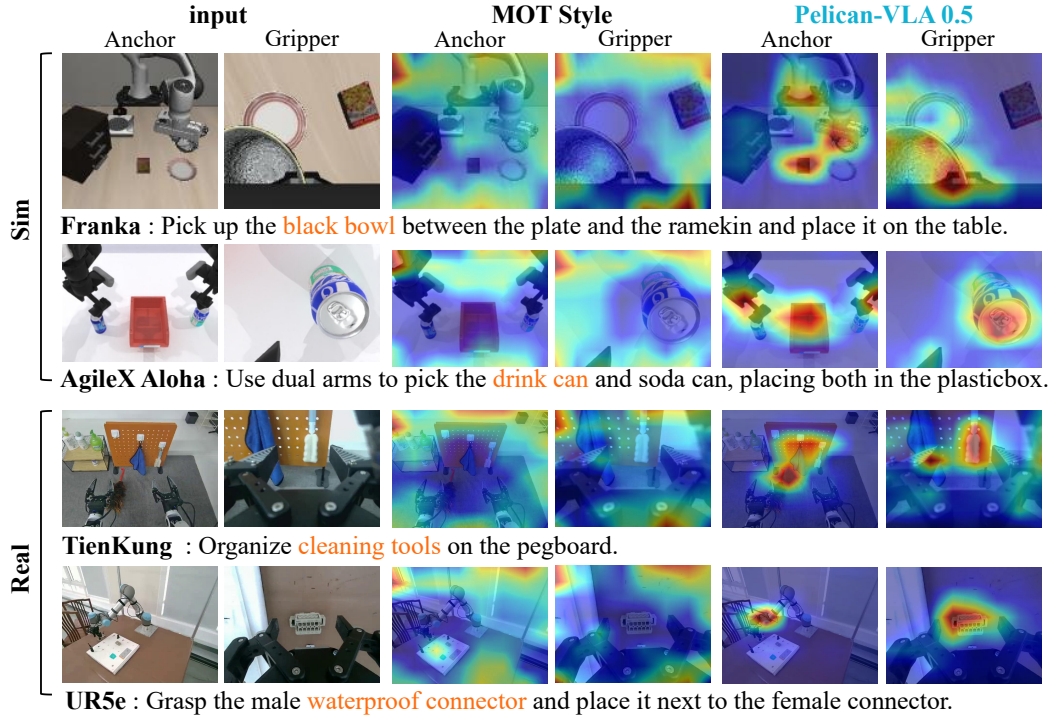


Figure 5: **Comparison of attention patterns between Pelican-VLA 0.5 and a dense MoT baseline under matched data, optimization, and parameter budgets.** Although both models achieve comparable action supervision objectives, the dense MoT baseline exhibits diffuse attention, whereas Pelican-VLA 0.5 forms concentrated object-centric attention patterns around instruction-relevant regions.

## 4. Analysis

In the zero-shot setting shown in Fig. 1, without task-specific fine-tuning and without object-, segmentation-, attention-, or reasoning-trace supervision, the attention maps of Pelican-VLA 0.5’s action pathway already highlight the instruction-relevant object and its contact region. This object-centric pattern is not directly supervised. It is also not explicitly imposed as an attention target. The goal of this section is therefore to identify where this representation comes from, how it develops during training, and why it does not yet fully translate into reliable zero-shot manipulation.

We combine qualitative attention visualization with an IoU-based grounding analysis. Following prior work on attention visualization in Transformers [1], we visualize where the action pathway attends over visual tokens. This protocol allows us to study whether the model routes perception through object-centric regions, and whether this routing changes with architecture, training objectives, pre-training progress, and fine-tuning.

### 4.1. Disentangling data from architecture

A first hypothesis is that the representation is already latent in the demonstrations, such that *any* sufficiently capable policy trained on them would acquire it; were this the case, the phenomenon would be uninformative about our design. We evaluate this hypothesis under a minimal intervention: holding the data, optimizer, schedule, and parameter budget fixed, we vary only the architecture, training a dense Mixture-of-Transformers (MoT) policy [8, 19] whose action expert attends *densely* to all upstream perception.

The two models differ substantially, as shown in Fig. 5. The dense MoT policy reaches a comparable training action loss, indicating that it can fit the demonstrations under the same data and compute budget. However, its attention remains diffuse and only weakly aligned with the target object. In contrast, Pelican-VLA 0.5 forms a concentrated, object-centric attention pattern. Since the training data and optimization setup are matched, the difference cannot be explained by the corpus alone. This suggests that the representation is induced by the architecture, and more specifically by how perception is routed to action.

### 4.2. Ablation

Having established that the effect is architectural, we next identify which component is responsible. Pelican-VLA 0.5 differs from a plain action model in several ways: it uses a future-frame generation loss, a slot-language contrastive loss, and the reasoning-slot bottleneck. The full model alone does not reveal which component causes the observed representation.

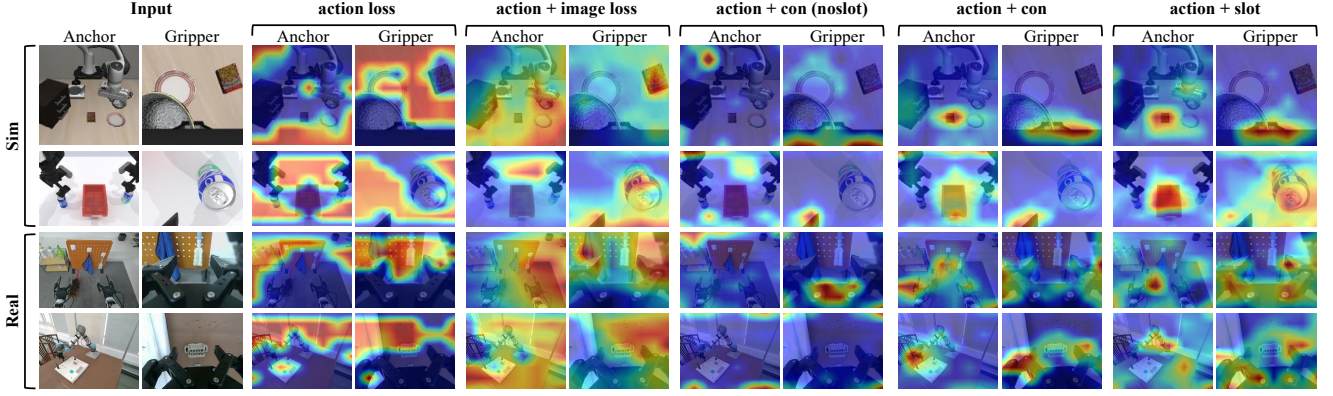


Figure 6: **Component-wise ablation of Pelican-VLA 0.5.** Adding each training objective individually shows that auxiliary losses alone do not induce object-centric attention; the object-centric pattern emerges when perception is routed through the slot bottleneck.

We therefore construct an ablation ladder that introduces one mechanism at a time while keeping the backbone, data, and training budget fixed. The ladder contains five configurations:

1. action loss only;
2. action loss + future-frame generation loss  $\mathcal{L}_{\text{gen}}$ ;
3. action loss + slot-language contrastive loss  $\mathcal{L}_{\text{task}}$ , applied to a dense model without slots;
4. action loss + the slot bottleneck + slot-language contrastive loss;
5. action loss + the slot bottleneck.

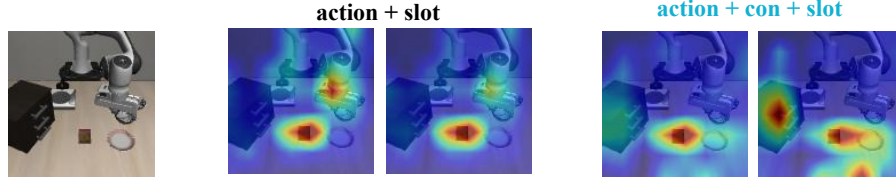
The ablation yields a clear result. The generation loss leaves the representation unchanged, and the contrastive loss imposed on a dense model likewise has no effect—both configurations leave the attention near chance. The representation emerges *only* at row 4, precisely when perception is routed through the slots, at which point it becomes pronounced. That a single mechanism accounts for the entire effect is notable; the evidence nonetheless is unambiguous: the auxiliary losses are not the cause, whereas the slot bottleneck is. Together with §4.1, this completes the attribution: the representation is induced specifically by routing perception through the narrow slot channel.

**Language-conditioned attention.** When examining intermediate checkpoints along this training ladder, we observed a phenomenon that cannot be fully explained by the slot bottleneck alone. In an early checkpoint, as shown in Fig. 7, the slot-only model tends to localize *an* object, often the visually dominant or frequently manipulated one, regardless of the instruction. After introducing the contrastive task loss, the same scene exhibits language-conditioned behavior: changing the instruction from *Pick up the black bowl between the plate and the ramekin and place it on the table.* to *Open the drawer of the black cabinet.* shifts the attention from the bowl to the cabinet, following the object specified by the language. This suggests that the contrastive loss does not create attentional focus by itself, but can make the slot content more conditional on language rather than only on visual salience.

However, this language-conditioned attention pattern appears mainly in early checkpoints and becomes less evident in later stages of training. We hypothesize that this degradation is related to noise in the training data, where a non-negligible portion of language instructions does not precisely match the executed task. As training proceeds, such language-action mismatch may weaken the language-conditioned representation signal. Therefore, we treat this observation as evidence that controllable representation can emerge from the combination of slots and  $\mathcal{L}_{\text{task}}$ , while also noting that its stability depends on the quality of language-action alignment in the data. In the final full-scale model, we will address this issue through stricter data curation and improved language-action alignment, so that language-conditioned representation can be preserved throughout training.

**Slot transferability across architectures.** To further examine whether the effect of reasoning slots is tied to the specific Pelican-VLA architecture, we also insert the same slot interface into a MoT-style architecture, as illustrated in Fig. 8. The resulting model exhibits a similar attention pattern: despite the change in backbone organization and action-prediction pathway, the slots consistently concentrate on regions related to the manipulated object and its contact area. This suggests that the slot bottleneck is not merely an artifact of our particular architecture, but can serve as a more general interface for routing task-relevant visual information toward action generation. While the strength and language selectivity of this behavior may still depend on the training objective and data quality, the MoT-based result indicates that slots provide a transferable mechanism for inducing manipulation-centric attention across different VLA designs.





**Original** : Pick up the **black bowl** between the plate and the ramekin and place it on the table.

**New** : Open the **drawer** of the black cabinet.

Figure 7: **Language-conditioned object grounding.** For the same visual scene, changing the instruction shifts the attention toward the newly specified target object. This instruction-dependent behavior becomes stronger with the slot-language contrastive objective, indicating improved alignment between language instructions and manipulation-centric attentions.

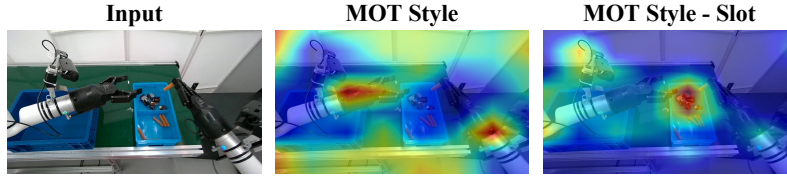


Figure 8: **Slot transferability across architectures.** We insert the same reasoning-slot interface into a MoT-style architecture to test whether the slot-induced attention pattern depends on the specific Pelican-VLA design. The resulting model still concentrates its attention on manipulation-relevant object regions, suggesting that reasoning slots can serve as a transferable interface for routing task-relevant visual information toward action generation across different VLA architectures.

| Region                      | IoU (50k) | IoU (500k) | $\Delta$ IoU | Spearman $\rho$      |
|-----------------------------|-----------|------------|--------------|----------------------|
| Manipulator (arm + gripper) | 0.360     | 0.350      | -0.010       | -0.43 ( $p = 0.21$ ) |
| Target object (bottle)      | 0.054     | 0.124      | +0.070       | 0.76 ( $p = 0.01$ )  |

Table 2: **Attention dynamics across pre-training checkpoints.** While manipulator attention remains stable and dominant, target object attention shows a strong monotonic increase, as evidenced by a significant positive Spearman correlation.

### 4.3. Attention dynamics over pre-training

To characterize how representation emerges during pre-training, we track attention through the slot bottleneck across checkpoints of the full model, as shown in Fig.9 and Table2.

As pre-training progresses, attention first becomes spatially concentrated and only later becomes aligned with the object specified by the instruction. This temporal ordering suggests a two-stage process: the model first learns to compress visual information into compact regions, and then learns to select the semantically relevant object among them.

Table 2 provides quantitative evidence for this trend. Manipulator-related regions remain a strong attention recipient throughout pre-training, with an IoU around 0.35 and no significant monotonic trend ( $\rho = -0.43$ ,  $p = 0.21$ ). In contrast, attention alignment with the target object increases significantly, with IoU rising from 0.054 to 0.124 and showing a strong positive monotonic trend across checkpoints ( $\rho = 0.76$ ,  $p = 0.01$ ). Thus, although the absolute attention mass remains partly biased toward the robot embodiment, the systematic change during pre-training is directed toward task-relevant objects.

This object-directed migration continues even after the action loss has largely plateaued, suggesting that representation is not merely a by-product of fitting the action objective. Instead, it reflects a slower reorganization of how task-relevant visual information is compressed through the slot bottleneck. Since no object-level or attention supervision is provided, this trajectory supports our view that manipulation-centric attention can emerge through pre-training under a slot-mediated architecture.

In addition, removing the slots after training does not fully eliminate the model’s ability to attend to the instruction-relevant object, indicating that the slot bottleneck helps internalize manipulation-centric attention into the shared backbone and allows the learned representation to persist as a property of the trained model itself.

### 4.4. Zero-shot vs fine-tuned representations

This report shows that Pelican-VLA 0.5 already develops manipulation-centric attention before any task-specific adaptation. A natural question is whether fine-tuning creates a new representation pattern for the target task, or instead builds on the representation that is already present. To examine this, we compare the zero-shot model and its fine-tuned counterpart on the same held-out task, using attention visualizations, representation maps, and representation-similarity measures.

As shown in Fig. 10, fine-tuning changes the representation pattern only marginally. The zero-shot and fine-tuned models attend to highly similar object regions, and their slot representations remain close under representation-similarity analysis. Their behavior, however, differs substantially. The zero-shot model often selects and approaches the correct object but fails at

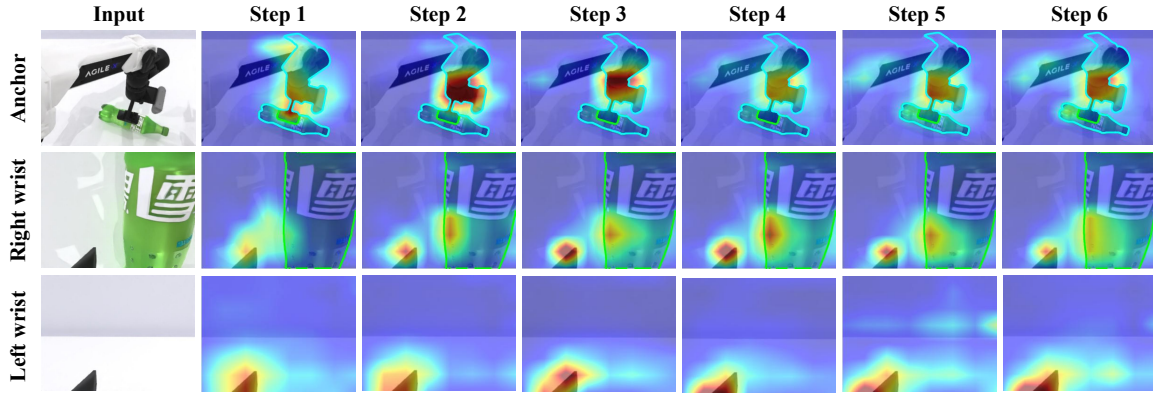


Figure 9: **The representation builds up over pre-training.** The slot attention starts diffuse and migrates onto the instruction-relevant object as training proceeds; the representation keeps rising after the action loss has saturated.

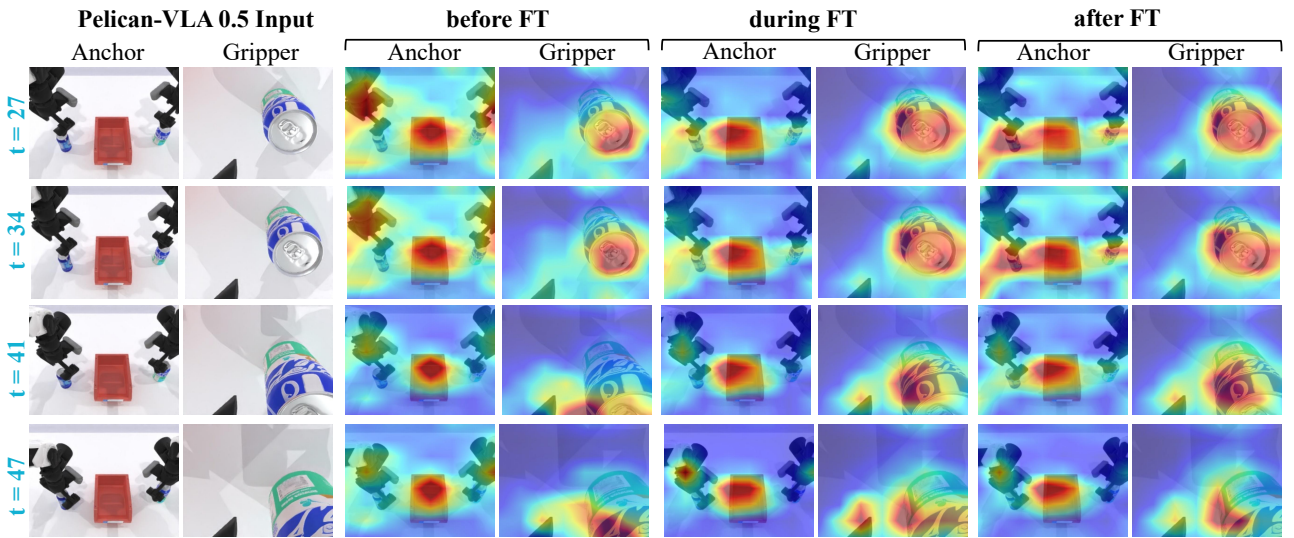


Figure 10: **Zero-shot and fine-tuned representations are highly similar.** Fine-tuning leaves the slot representation and representation map largely unchanged, yet task success increases substantially. This suggests that the grounded representation is already present before adaptation, while fine-tuning mainly improves the mapping from representation to action.

Table 3: Preservation of manipulation-centric attentions during fine-tuning. We compare target attention alignment and attention similarity with respect to the zero-shot model.

| Stage     | Target Attention IoU | Attention Similarity |
|-----------|----------------------|----------------------|
| Before FT | 0.124                | 1.000                |
| During FT | 0.134                | 0.932                |
| After FT  | 0.127                | 0.928                |

fine-grained control, whereas the fine-tuned model executes the task reliably.

To quantify this observation, we measure target attention IoU, attention similarity, and slot representation similarity between the zero-shot model and fine-tuned checkpoints, as summarized in Table 3. For the similarity metrics, we compute cosine similarity between temporally aligned attention maps and slot representations from the zero-shot and fine-tuned models. The similarity score measures the consistency of the underlying representation, where a higher value indicates that fine-tuning preserves the corresponding attention pattern or latent representation. The target attention IoU remains stable throughout fine-tuning, while the attention similarity to the zero-shot model remains above 0.9, indicating that task adaptation preserves the object-centric attention pattern and does not substantially alter where the model attends. Meanwhile, the slot representations undergo moderate adaptation while remaining substantially aligned with the zero-shot representation, suggesting that fine-tuning refines the existing manipulation-centric attention to better support action generation rather than replacing it from scratch.

This result supports the representation-to-action gap introduced earlier. The manipulation-centric attention needed to iden-



tify *what* to act upon is largely present before fine-tuning. Fine-tuning does not primarily create this grounding from scratch; rather, it adapts and translates this grounded representation into executable actions.

## 5. Conclusion

This report presents **Pelican-VLA 0.5**, a unified VLA model built around a compact set of learnable *Reasoning Slots*. Our main finding is that routing perception through this slot-mediated interface induces manipulation-centric attention before task-specific adaptation. Without object labels, segmentation masks, attention supervision, or reasoning traces, the action pathway learns to concentrate on the instruction-relevant object and its contact region. Controlled comparisons and ablations further show that this behavior is not explained by training data alone. Instead, it is primarily shaped by the slot-mediated route from perception to action, while the slot-language objective makes the representation more controllable by language.

At the same time, Pelican-VLA 0.5 makes clear that zero-shot representation is not the same as zero-shot manipulation. In unseen scenes and unseen robot embodiments, the pre-trained model often identifies and approaches the correct object, but its strict zero-shot success rate remains low. After fine-tuning on RoboTwin, the model achieves state-of-the-art average success among open-source VLA models on this benchmark, while its attention maps and slot representations remain highly similar to those of the zero-shot checkpoint. This suggests that fine-tuning does not primarily create representation from scratch. Rather, it strengthens the mapping from an already-formed manipulation-centric attention to executable actions.

We therefore view Pelican-VLA 0.5 as an intermediate stage between visual representation and practical zero-shot manipulation. The model has begun to answer the question of *what* to act upon, but has not yet fully solved *how* to act upon it across new scenes and embodiments. We refer to this remaining challenge as the *representation-to-action gap*. It is most visible in fine-grained manipulation stages such as stable grasping, contact timing, and precise placement.

We attribute this gap mainly to data scale and action representation. Pelican-VLA 0.5 is trained on only about 2400 hours of heterogeneous manipulation data, and the data itself remains partially incomplete and noisy. Moreover, the model uses joint-position actions, which are more embodiment-specific than end-effector pose representations and therefore less favorable for cross-embodiment transfer. These limitations likely prevent the grounded representation from being fully converted into reliable zero-shot action competence.

Our next step is to scale both data and action experience. We plan to release a stronger version trained on approximately 7000 hours of manipulation data, superseding the current 2400-hour checkpoint. Beyond this, we will continue curating large-scale manipulation data and improving action parameterization, with the goal of turning manipulation-centric attention into robust zero-shot control. To support reproducibility and further research, we will release the code, model weights, and attention-visualization tools.

## References

- [1] S. Abnar and W. Zuidema. Quantifying attention flow in transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [2] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, D. Dworakowski, J. Fan, M. Fenzi, F. Ferroni, S. Fidler, D. Fox, S. Ge, Y. Ge, J. Gu, S. Gururani, E. He, J. Huang, J. Huffman, P. Janaty, J. Jin, S. W. Kim, G. Klár, G. Lam, S. Lan, L. Leal-Taixe, A. Li, Z. Li, C.-H. Lin, T.-Y. Lin, H. Ling, M.-Y. Liu, X. Liu, A. Luo, Q. Ma, H. Mao, K. Mo, A. Mousavian, S. Nah, S. Niverty, D. Page, D. Paschalidou, Z. Patel, L. Pavao, M. Ramezanali, F. Reda, X. Ren, V. R. N. Sabavat, E. Schmerling, S. Shi, B. Stefaniak, S. Tang, L. Tchapmi, P. Tredak, W.-C. Tseng, J. Varghese, H. Wang, H. Wang, H. Wang, T.-C. Wang, F. Wei, X. Wei, J. Z. Wu, J. Xu, W. Yang, L. Yen-Chen, X. Zeng, Y. Zeng, J. Zhang, Q. Zhang, Y. Zhang, Q. Zhao, and A. Zolkowski. Cosmos world foundation model platform for physical ai, 2025.
- [3] AgiBot-World-Contributors, Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, S. Gao, X. He, X. Hu, X. Huang, S. Jiang, Y. Jiang, C. Jing, H. Li, J. Li, C. Liu, Y. Liu, Y. Lu, J. Luo, P. Luo, Y. Mu, Y. Niu, Y. Pan, J. Pang, Y. Qiao, G. Ren, C. Ruan, J. Shan, Y. Shen, C. Shi, M. Shi, M. Shi, C. Sima, J. Song, H. Wang, W. Wang, D. Wei, C. Xie, G. Xu, J. Yan, C. Yang, L. Yang, S. Yang, M. Yao, J. Zeng, C. Zhang, Q. Zhang, B. Zhao, C. Zhao, J. Zhao, and J. Zhu. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems, 2025.
- [4] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, W. Ge, Z. Guo, Q. Huang, J. Huang, F. Huang, B. Hui, S. Jiang, Z. Li, M. Li, M. Li, K. Li, Z. Lin, J. Lin, X. Liu, J. Liu, C. Liu, Y. Liu, D. Liu, S. Liu, D. Lu, R. Luo, C. Lv, R. Men, L. Meng, X. Ren, X. Ren, S. Song, Y. Sun, J. Tang, J. Tu, J. Wan, P. Wang, P. Wang, Q. Wang, Y. Wang, T. Xie, Y. Xu, H. Xu, J. Xu, Z. Yang, M. Yang, J. Yang, A. Yang, B. Yu, F. Zhang, H. Zhang, X. Zhang, B. Zheng, H. Zhong, J. Zhou, F. Zhou, J. Zhou, Y. Zhu, and K. Zhu. Qwen3-vl technical report, 2025.
- [5] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. GR00T N1: An open foundation model for generalist humanoid robots, 2025.
- [6] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky.  $\pi_{0.5}$ : a vision-language-action model with open-world generalization, 2025.
- [7] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky.  $\pi_0$ : A vision-language-action flow model for general robot control, 2026.
- [8] J. Cai, Z. Cai, J. Cao, Y. Chen, Z. He, L. Jiang, H. Li, H. Li, Y. Li, Y. Liu, Y. Lu, Q. Lv, H. Ma, J. Pang, Y. Qiao, Z. Qiu, Y. Shen, X. Shi, Y. Tian, B. Wang, H. Wang, J. Wang, T. Wang, X. Wei, C. Wu, Y. Xie, B. Xing, Y. Yang, Y. Yang, Q. Yu, F. Yuan, J. Zeng, J. Zhang, S. Zhang, S. Zhang, Z. Zhaxi, B. Zhou, Y. Zhou, Y. Zhou, H. Zhu, Y. Zhu, and Y. Zhu. Internvla-a1: Unifying understanding, generation and action for robotic manipulation, 2026.
- [9] H. Chen, J. Guo, B. Wang, T. Zhang, X. Huang, B. Zheng, Y. Hou, C. Tie, J. Deng, and L. Shao. Goal-vla: Image-generative vlms as object-centric world models empowering zero-shot robot manipulation, 2026.
- [10] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, W. Deng, Y. Guo, T. Nian, X. Xie, Q. Chen, K. Su, T. Xu, G. Liu, M. Hu, H. ang Gao, K. Wang, Z. Liang, Y. Qin, X. Yang, P. Luo, and Y. Mu. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025.
- [11] S. Community. Starvla: A lego-like codebase for vision-language-action model developing, 2026.
- [12] X. Jia, B. Yang, Z. Ge, X. Nie, Y. Zhou, C. Fan, Y. Li, Y. Chai, C. Jing, Z. Liang, Q. Bu, H. Cao, C. Wu, Q. Li, Z. Yang, C. Zhang, H. Li, Z. Wu, J. Yan, and Y.-G. Jiang. Guidedvla: Specifying task-relevant factors via plug-and-play action attention specialization, 2026.
- [13] T. Jiang, T. Yuan, Y. Liu, C. Lu, J. Cui, X. Liu, S. Cheng, J. Gao, H. Xu, and H. Zhao. Galaxea open-world dataset and g0 dual-system vla model, 2025.
- [14] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, et al. Droid: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.



- [15] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [16] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *11th International Conference on Learning Representations, ICLR 2023*, 2023.
- [17] M. Liu, B. Jia, J. Huang, J. Zhang, and S. Huang. Lara: Latent action representation alignment for vision-language-action models, 2026.
- [18] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: A diffusion foundation model for bimanual manipulation, 2024.
- [19] Z. Liu, J. Liu, H. Chen, J. Yu, Z. Guo, C. Hou, C. Gu, X. Mi, R. Zhang, K. Wu, Z. Che, J. Tang, P.-A. Heng, and S. Zhang. Lasto: Latent spatio-temporal chain-of-thought for robotic vision-language-action model, 2026.
- [20] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024.
- [21] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903, 2024.
- [22] Open-X-Humanoid. Tienkung humanoid robot platform. GitHub repository, 2025. Robot URDF description, mesh files, and platform resources for the TienKung humanoid. Accessed July 1, 2026.
- [23] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model, 2025.
- [24] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.
- [25] Y. Tian, Y. Yang, Y. Xie, Z. Cai, X. Shi, N. Gao, H. Liu, X. Jiang, Z. Qiu, F. Yuan, Y. Li, P. Wang, J. Cai, J. Zeng, H. Dong, and J. Pang. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy, 2025.
- [26] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [27] Q. Wang, M. Li, J. Guan, J. Ye, S. Xie, Y. Liu, J. Chen, Z. Liang, J. Zhang, X. Hu, X. Huang, P. Lin, J. Lin, D. Liu, S. Bai, J. Zhou, J. Zhang, H. Yuan, G. Zhou, H. Yin, Y. Wang, Y. Huang, Z. Lei, W. Peng, D. Chen, Y. Zheng, J. Fan, X. Zhuang, X. Zhou, H. Li, A. Chen, T. Zhang, X. Liu, Y. Sun, R. Chen, Z. Li, C. Lü, Z. Yang, T. Yu, and X. Chen. Qwen-vla: Unifying vision-language-action modeling across tasks, environments, and robot embodiments, 2026.
- [28] K. Wu, C. Hou, J. Liu, Z. Che, X. Ju, Z. Yang, M. Li, Y. Zhao, Z. Xu, G. Yang, S. Fan, X. Wang, F. Liao, Z. Zhao, G. Li, Z. Jin, L. Wang, J. Mao, N. Liu, P. Ren, Q. Zhang, Y. Lyu, M. Liu, H. Jingyang, Y. Luo, Z. Gao, C. Li, C. Gu, Y. Fu, D. Wu, X. Wang, S. Chen, Z. Wang, P. An, S. Qian, S. Zhang, and J. Tang. Robomind: Benchmark on multi-embodiment intelligence normative data for robot manipulation. In *Robotics: Science and Systems XXI*, RSS2025. Robotics: Science and Systems Foundation, 2025.
- [29] W. Wu, F. Lu, Y. Wang, S. Yang, S. Liu, F. Wang, Q. Zhu, H. Sun, Y. Wang, S. Ma, Y. Ren, K. Zhang, H. Yu, J. Zhao, S. Zhou, Z. Qiu, H. Xiong, Z. Wang, Z. Wang, R. Cheng, Y.-L. Li, Y. Huang, X. Zhu, Y. Shen, and K. Zheng. A pragmatic vla foundation model, 2026.
- [30] X. Xu, H. Li, J. Ye, Y. Chen, J. Zeng, X. Chen, L. Xu, D. Lin, W. Li, and J. Pang. Futurevla: Joint visuomotor prediction for vision-language-action model, 2026.
- [31] Y. Yang, S. Zeng, T. Lin, X. Chang, D. Qi, J. Xiao, H. Liu, R. Chen, Y. Chen, D. Huo, F. Xiong, X. Wei, Z. Ma, and M. Xu. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning, 2026.
- [32] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, et al. World action models are zero-shot policies, 2026.
- [33] L. Zha, A. J. Hancock, M. Zhang, T. Yin, Y. Huang, D. Shah, A. Z. Ren, and A. Majumdar. Lap: Language-action pre-training enables zero-shot cross-embodiment transfer, 2026.
- [34] H. Zhang, L. Xiang, H. Lin, Z. Huang, M. Wang, D. Zhong, Y. Dong, Y. Wu, Y. Rao, D. Zhang, W. He, L. Chen, K. Huang, J. Chen, S. Su, X. Yu, Z. Wang, C. Zhu, X. Teng, Y. Guo, Y. Zhang, Y. Liu, R. Wang, Z. Lu, H. Hu, and Z. Zhang. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack, 2026.



- [35] T. Zhang, Z. Yuan, D. Chi, P. Liu, D. Li, K. Hu, L. Zhang, J. Nie, Z. Wei, Z. Chen, Y. Tang, J. Li, Z. Xiang, M. Li, T. Luo, H. Wan, A. Li, L. Zhai, Z. Zhan, X. Bai, J. Cai, P. Cao, K. Chen, S. Chen, Y. Dai, S. Di, Y. Gong, C. Gui, Y. Guo, P. Hao, Q. He, H. Huang, K. Huang, Z. Huang, S. Jin, Y. Jin, A. Li, D. Li, J. Li, R. Li, Y. Li, Y. Li, J. Liang, F. Liu, J. Long, M. Luo, X. Pan, H. Shen, X. Tian, D. Wang, S. Wang, J. Xiong, H. Xu, W. Xu, Z. Yu, H. Zhang, J. Zhang, L. Zhao, C. Zhou, N. Duan, Y. Zhuang, and L. Lin. Joyai-ra 0.1: A foundation model for robotic autonomy, 2026.
- [36] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025.
- [37] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.





## 6. Contributions

Our contributors are organized based on their roles and magnitude of contribution.

### 6.1. Core Contributors

Data, model, code and pre-training: Zeyuan Ding

Code, real-world and simulation experiments: Wenhai Liu

Robot deployment and attention visualization: Yang Xu, Jiayu Hu

### 6.2. Contributors

Yinda Chen, Yi Zhang

### 6.3. Tech Lead

Yong Dai, Zeyuan Ding

### 6.4. Corresponding Authors

Jian Tang, Xiaozhu Ju